

Design Choices

Overall Design Choices:

Throughout the designing process, we endeavoured to ensure that our app's design was functional, accessible, and pleasing to the eye. To do this, we made sure to follow Google's guidelines for accessibility and theming, as well as minimising the amount shown on the screen at once so as to not overwhelm the user. Each page was also frequently tested with the Android Accessibility scanner and TalkBack upon the implementation of new features and changes to the UI, ensuring that they were accessible to all, and fixing any changes that weren't in line with these recommendations.

Our theme colours, used throughout the entire app, were picked using the [Material Theme Builder](#), with the base colour being a similar blue to the blue used throughout our wire-frames. This allowed us to use a custom theme for both light and dark mode and high contrast of either of these, as well as ensuring that our colours were suitably contrasted against each other for accessibility and readability purposes.

Font sizes throughout the app were set in *sp* (scale-independant pixels), meaning that they can adapt to the font size that the user has chosen in their phone's system. As a general rule, the following sizes were observed throughout:

Element	Size (sp)	Example
Page titles	22	MyBasket at the top of MainMenuList.kt
TextField inputs	16	User entry into username TextField in Login.kt
Textfield labels and placeholders	14	"Enter username" TextField placeholder in Login.kt
Dialog titles	16, medium weight	"Add List" dialog title
Dialog TextButtons and bodies	14	"Add" button
List and item names	16, medium weight	User-inputted name of item in Pantry.kt
List and item attributes	14	Quantity
Error messages	14	Invalid email error in Register.kt

Accessibility was checked throughout all pages in the app. The colours chosen for both the light and dark mode were chosen both for their aesthetic strengths, as well

as for their contrast against each other. All elements were checked using Accessibility Scanner to ensure that they had a minimum contrast ratio of 4.5:1, making them suitable for those with visual impairments such as colourblindness. All Buttons, IconButton, and TextButtons were given a minimum default size of 48.dp to ensure that they are accessible to those who may have dexterity impairments. Items were also given content descriptions where applicable, ensuring that the page is accessible to users using screen readers.

Layout and Positioning:

Throughout the design of the app, we generally worked to have the display be symmetrical and consistent across each page, particularly across pages with similar functionality. We did this as we felt it lessened the learning curve for new users of the app (e.g. if a user can use the Lists screen, they can easily learn the Pantry, and List Items pages due to their similarity), as well as this practice being generally aesthetically pleasing.

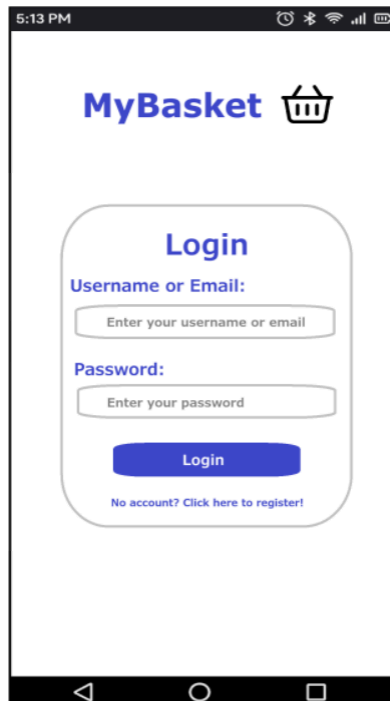
This centered design also has a functional reason, that being that the user will generally be able to reach each item easily with one hand. This allows the user to use the app easily when not focused entirely on it, or if they have their hands full at the time (which may often be the case while shopping). Keeping each item on the screen centred allows for easy reach, regardless of whether the user is left-handed or right-handed, or if the user doesn't have an extra hand free.

We generally opted for our layout to be as simple as possible, with the only "complex" layout being our calendar screen. Each page had a set purpose and stuck to that purpose, and its design would revolve around that purpose. We have support for both English and Spanish in the app, but we also made sure to use icons throughout the app for navigation and confirmation, so as to allow for recognition of each button's general purpose on user recognition alone, which can speed up the learning process.

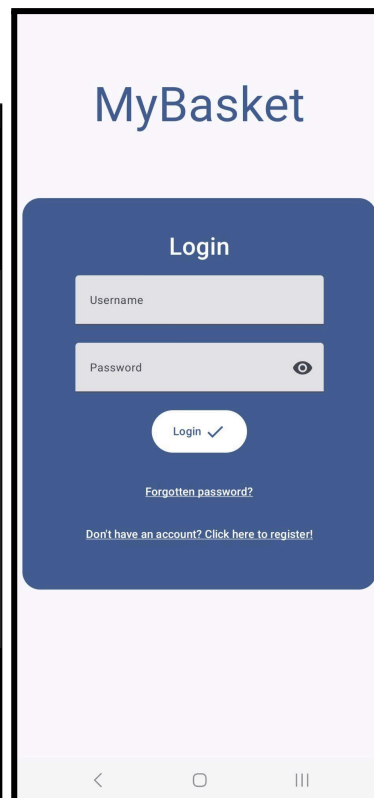
Any icons we used throughout were chosen due to their recognisable use and purpose. The plus, back, forward, settings, eye, and kebab menu icons are all commonly used icons, and therefore the vast majority of users will be familiar with their meaning and usage. The icons used for the bottom bar were chosen as they are a good visual shorthand for what each IconButton will lead to, especially alongside their respective labels.

Login:

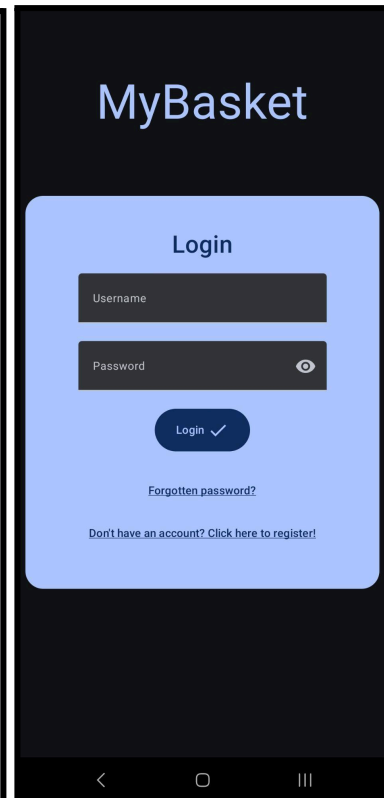
Wire-frame



Light mode



Dark mode



Our login page changed very little structure-wise between the wire frame and the completed app, with the main difference actually being the colours chosen. Its main purpose is to allow a user to login and gain access to their lists, pantry, and calendar. It also allows a user who has forgotten their password to reset it using their email, or a user without an account to navigate to a registration page.

The TextFields that take in the Username and Password are connected to the Room database and when the Login button is pressed, the database is checked to see whether the user exists and if a user is found with that username and password then the user is logged in and brought to MainMenuList.kt. If no user is found then an error is shown and the user stays on the Login.kt page. There is also an IconButton that allows users to show or hide their password.

The first TextButton is for resetting the user's password if they have forgotten it. It works by presenting the user with a Dialog, wherein they must first enter their username and email and, provided these are correct, they can then enter a new password.

The second TextButton is for registering an account, and will bring the user to Register.kt.

Register:

Wire-frame

The wireframe shows the Register page for the MyBasket app. At the top, there's a status bar with the time 5:13 PM and various icons. Below that is the MyBasket logo and a shopping cart icon. The main content area is titled 'Register' and contains four input fields: 'Email' (with placeholder 'Enter your email'), 'Username' (with placeholder 'Enter your username'), 'Password' (with placeholder 'Enter your password'), and 'Re-enter your password' (with placeholder 'Re-enter your password'). Each field is preceded by its respective label. Below the input fields is a blue 'Register' button. At the bottom, there's a link that says 'Already have an account? Click here to login'.

Light mode

The light mode version of the Register page features a light blue background. The title 'MyBasket' is at the top in a dark blue font. Below it, the 'Register' title is centered. The input fields are white with light blue borders and labels. The 'Register' button is white with a blue checkmark. The link at the bottom is 'Already have an account? Click here to login!'.

Dark mode

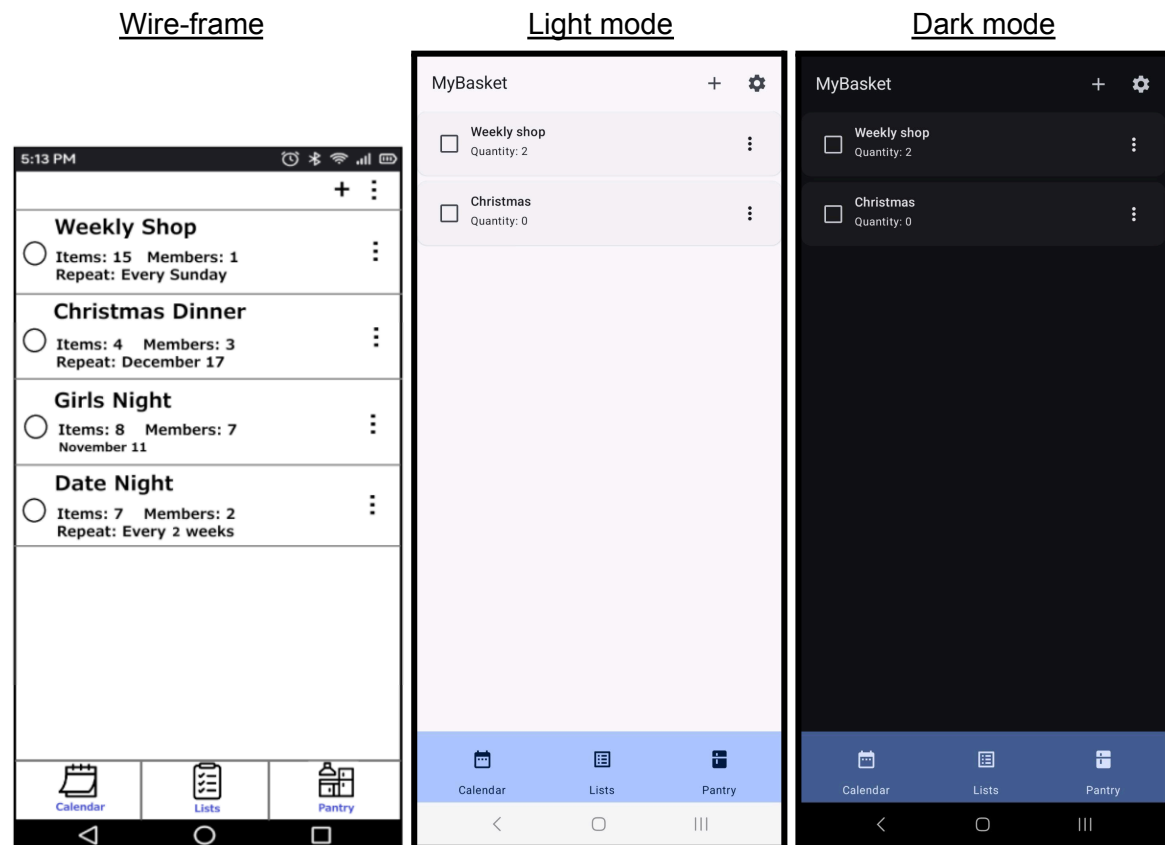
The dark mode version of the Register page has a dark blue background. The title 'MyBasket' is at the top in a light blue font. Below it, the 'Register' title is centered. The input fields are dark blue with light blue borders and labels. The 'Register' button is dark blue with a light blue checkmark. The link at the bottom is 'Already have an account? Click here to login!'.

Our register page had no changes structure-wise between the wire frame and the completed app, exclusively colour changes being made. Its main purpose is to allow a user to register an account through which they can login to create lists, a pantry, and a calendar. It also allows users who already have an account to navigate to the Login.kt screen.

The TextFields that take in the Email, Username, Password, and repeated Password are validated using validation functions in a validation file. The main validation is the email is correctly formatted, the username isn't already in use and is of the correct length, and that the two passwords match. If one of these checks fails then the user is informed via an error. When the Register button is pressed and none of them failed, the details are then added to the database and the user is redirected to the Login.kt page.

The TextButton is for logging into an account, and will bring the user to Login.kt.

Main Menu Lists:



The Main Menu is also largely the same as the original wire-frame design, with only a few changes such as the removal of the “Members” and “Repeat” counts, as these didn’t get implemented in the final app. The main function of the page is to allow the user to create, delete, show and check off shopping lists.

Each list on the screen is gotten from the Room Database is an ElevatedCard, displaying the lists name and the quantity of items within. Each list can be tapped, bringing the user to a ListItems.kt screen for that list.

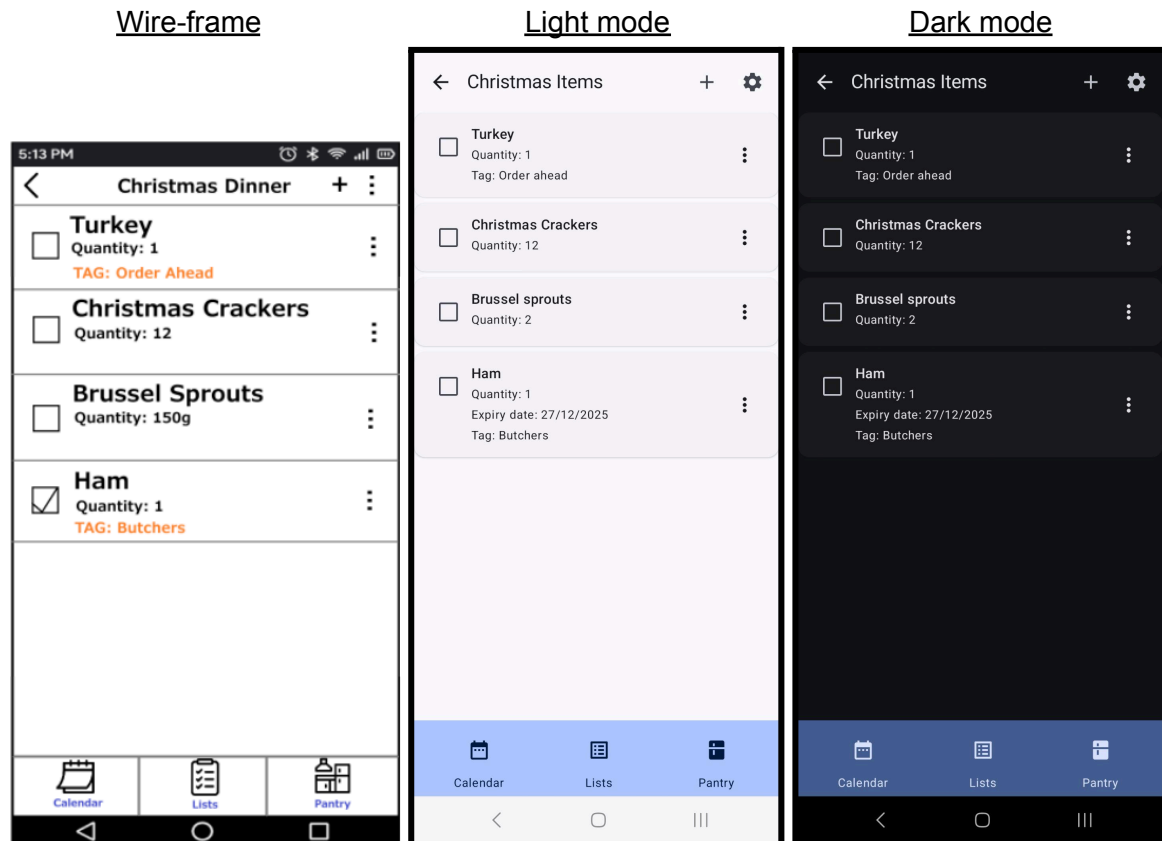
The plus IconButton in the top app bar allows the user to create a new list. It begins by showing a Dialog with a TextField in which the user can enter their new lists name. When the “Add” button is pressed, the inputted name is trimmed and checked to make sure it isn’t empty, then a new list is created and added to the screen.

The settings IconButton in the top app bar brings the user to AppSettings.kt.

Each list has a kebab menu, which displays a Dropdown menu when pressed which contains two TextButtons, Settings and Delete. When Settings is pressed the user is redirected to ListSettings.kt. When Delete is pressed, the list is deleted.

The bottom bar at the bottom of the screen contains three IconButton, Calendar, Lists, and Pantry. When the user clicks any of these they are directed to Calendar.kt, MainMenuLists.kt, and Pantry.kt respectively.

List Items:



The List Items page is also quite similar to its wireframe and the Main Menu page, with the only major difference being that the Tag on each item isn't orange as it is in the wire-frame, and the top bar displays [list name] Items and has a back button to return the user to MainMenuLists.kt.

Each item on the screen obtained from the ShoppingListViewModel is an ElevatedCard, displaying the item's name and details (if they have been filled in). Each item can be tapped, bringing the user to the ItemInfo.kt screen for that item. There is also a checkbox on the left of each item, which adds the item to the pantry when checked and removes it if unchecked.

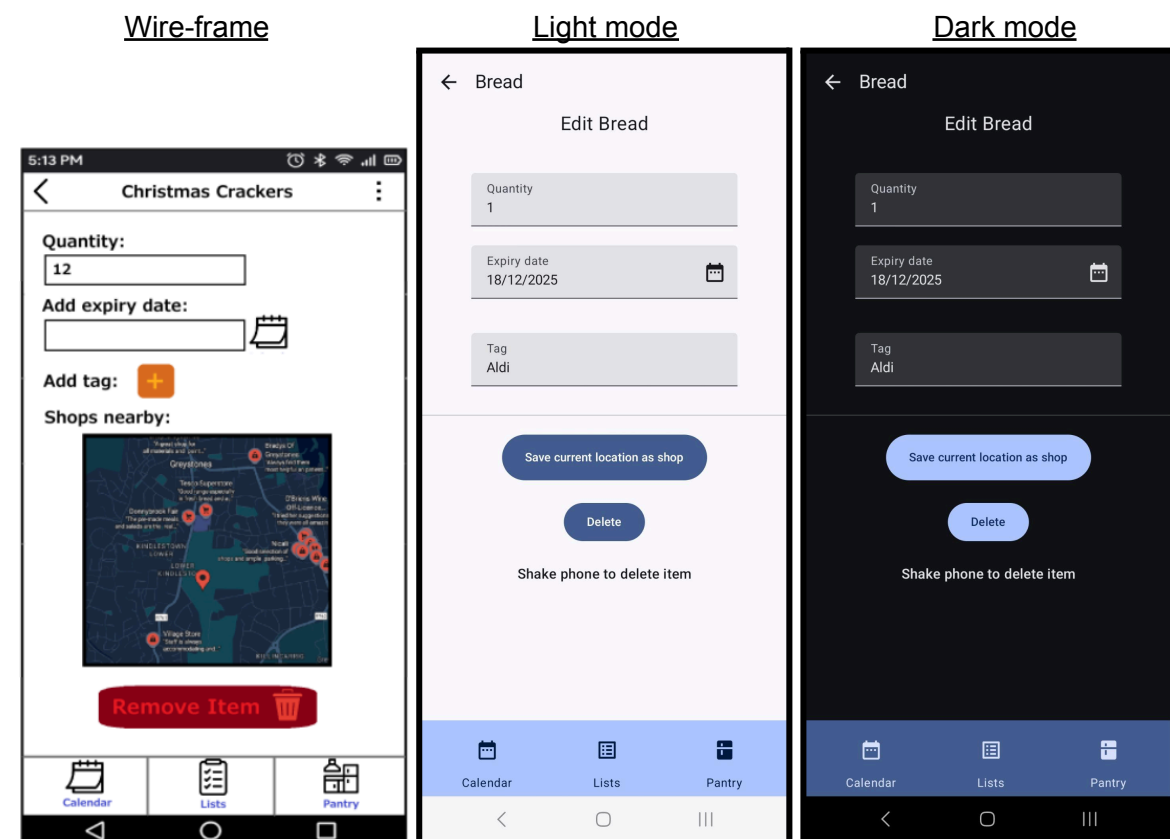
Each item has a kebab menu, which displays a Dropdown menu when pressed which contains two TextButtons, Settings and Delete. When Settings is pressed the user is redirected to ItemInfo.kt. When Delete is pressed, the item is deleted from the list.

The plus IconButton in the top app bar allows the user to create a new item. It begins by showing a Dialog with a TextField in which the user can enter their new lists name or they can choose from items in other lists or the pantry. When the “Add” button is pressed, the inputted name or choice is trimmed and checked to make sure it isn't empty or doesn't exist already, then a new item is created and added to the screen.

The settings IconButton in the top app bar brings the user to AppSettings.kt.

The bottom bar at the bottom of the screen contains three IconButton, Calendar, Lists, and Pantry. When the user clicks any of these they are directed to Calendar.kt, MainMenuLists.kt, and Pantry.kt respectively.

Item Info:



The Item Info page is probably one of the most different to its wireframe, with the map being unable to be displayed as it is in the wire-frame due to issues with paying for and getting an API key for google maps, and our revised choice to only include one tag per item.

The TextFields that take in the Quantity, Expiry Date and Tag are connected to the Room database and when they are altered, a Save Changes button appears. When this is pressed, the data that was entered or changed is entered into the database. The Expiry Date TextField can have its data entered either via a keyboard, wherein

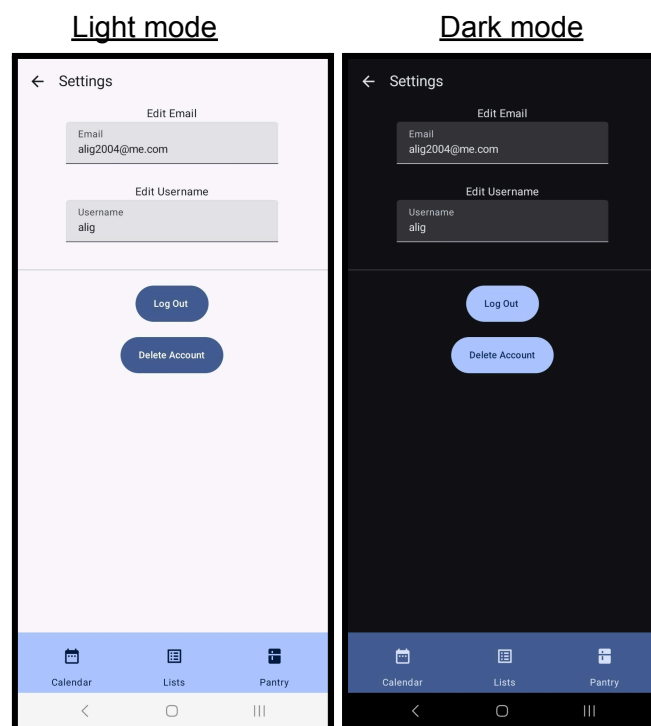
the input is checked to ensure that the date is valid, or by a DatePicker Dialog, which is called when the user presses the IconButton at the end of the TextField. We felt that having the option of either a date picker or being able to type in the data allowed for users to choose their dates much faster, with the date picker being perfect for dates closer to the current date, and the TextField being better for dates further away.

To make up for the lack of a map, we instead implemented a way for users to save their current location as a shop for the item. The user can then click a button that appears when a location has been saved to be redirected to google maps, set with that location as the destination.

We also implemented two different ways of deleting items. The user can either press the Delete Button, or they can shake the phone. To ensure no accidental deletions, a Dialog appears when the phone is shaken to check that the user intended to delete this item. Upon deletion of the item, the user will be returned to the previous page they were on.

The top bar has a back button to return the user to ListItems.kt. The bottom bar at the bottom of the screen contains three IconButton, Calendar, Lists, and Pantry. When the user clicks any of these they are directed to Calendar.kt, MainMenuLists.kt, and Pantry.kt respectively.

App Settings:



The App Settings screen was not a screen we had originally intended to add, and as such we have no wire-frame with which to compare it to. We chose to add it as we felt the users needed a way to alter their details, and a way to allow them to log out or delete their account.

The TextFields that take in the Email and Username are connected to the Room database and when they are altered, a Save Changes button appears. When this is pressed, the data that was entered or changed is checked to ensure that they are valid, and if this is the case it is updated in the database.

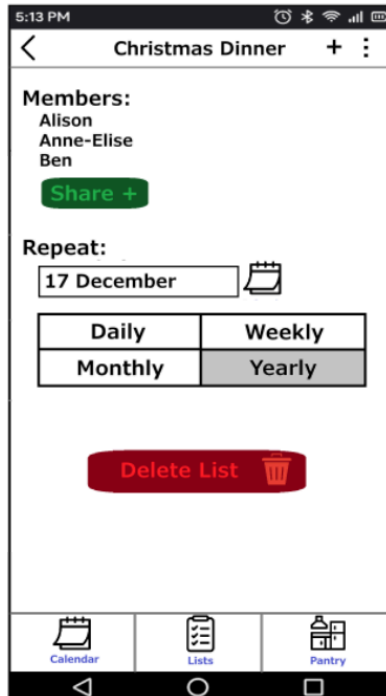
The Logout Button can be used by the user to log out of their account. When it is pressed a Dialog appears, confirming with the user that they want to log out of their account. If the user clicks the “Log Out” TextButton, the user is redirected to Login.kt. If the user clicks the “Cancel” TextButton, the Dialog is dismissed.

The Delete Account Button can be used by the user to delete their account and all lists associated with it. When it is pressed a Dialog appears, confirming with the user that they want to log out of their account. If the user clicks the “Delete” TextButton, the account and its data is removed from the database and the user is redirected to Login.kt. If the user clicks the “Cancel” TextButton, the Dialog is dismissed.

The top bar has a back button to return the user to the previous page. The bottom bar at the bottom of the screen contains three IconButton, Calendar, Lists, and Pantry. When the user clicks any of these they are directed to Calendar.kt, MainMenuLists.kt, and Pantry.kt respectively.

List Settings:

Wire-frame



Light mode



Dark mode



The List Settings screen is quite different from its wireframe, as we ended up deciding against various elements that would have been included in it.

As we removed the idea of repeating lists, we had no reason for the input of a date or time frame. We also removed the button to delete lists as it would have overcomplicated our DAO layouts.

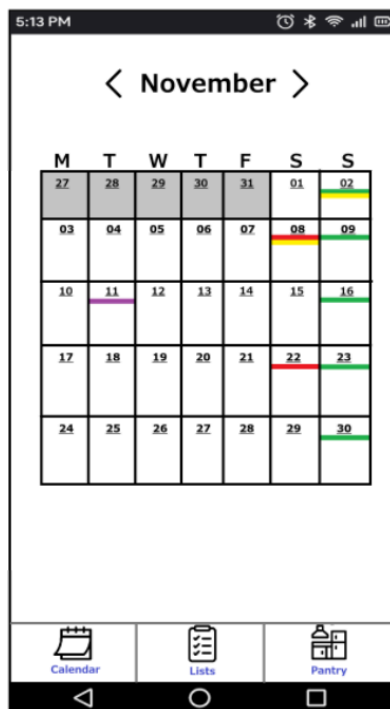
We decided against listing the names of “Members” as our current way of sharing lists doesn’t allow for the list data to be automatically updated across different devices, something which we had originally planned to do but couldn’t due to time constraints. We instead implemented a way for the user to share their lists via email. When the “Share list by email” Button is pressed, a Dialog is shown which has a TextField wherein the user can input the email of the person they’d like to share the list with. When this is done, if the user presses “Send” then they are redirected to their phone's email app, with a new draft containing the input email as the recipient, the list title as the subject, and each item in the lists name and details in the body section. If they click cancel, the Dialog is removed from the screen.

The top bar has a back button to return the user to the previous page. The bottom bar at the bottom of the screen contains three IconButton, Calendar, Lists, and

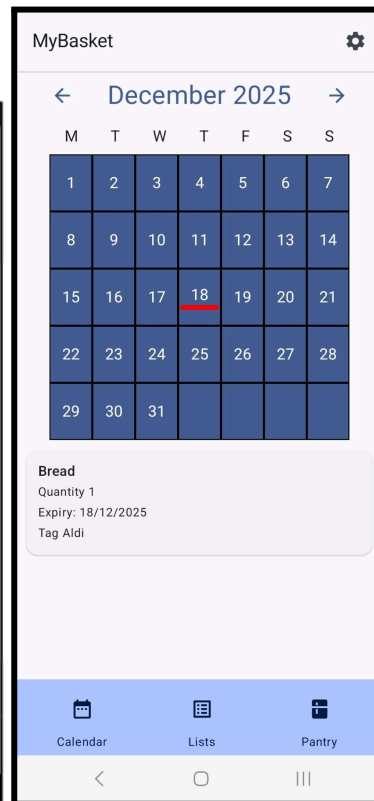
Pantry. When the user clicks any of these they are directed to Calendar.kt, MainMenuLists.kt, and Pantry.kt respectively.

Calendar:

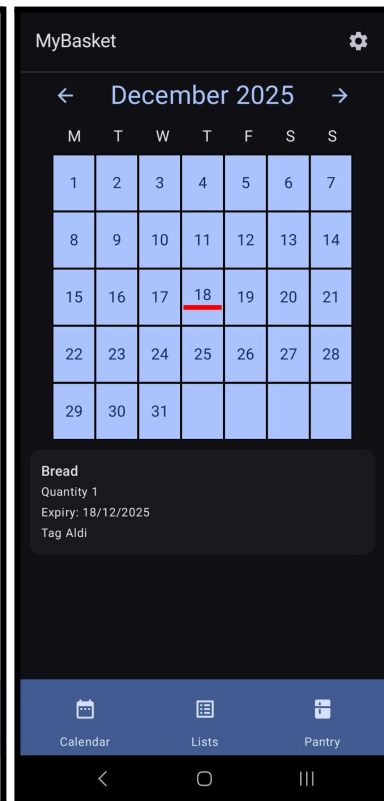
Wire-frame



Light mode



Dark mode



The Calendar page only changed slightly from the wire-frame, largely with the decision to get rid of repeating lists, and the choice to allow items expiring on a clicked day to show below.

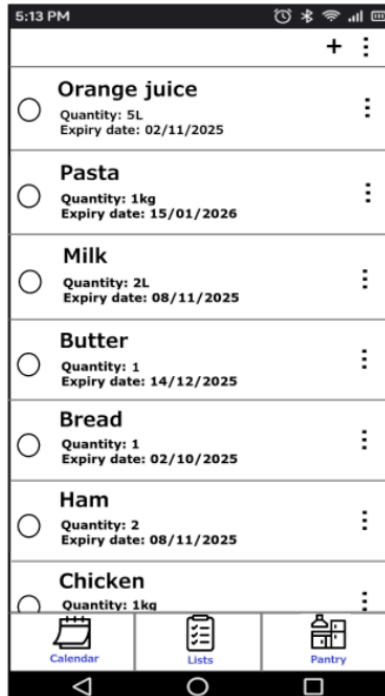
When it is navigated to, the calendar will automatically show the user the current month, with any days where pantry items are set to expire having a red line across them. The user can press on the arrow buttons on either side of the month and year at the top to navigate to the previous or next month.

If the user presses on a day cell, any items set to expire on that day, obtained from the Room database, will appear as ElevatedCards below the calendar itself. These cards inform the user of the item and its details, and will redirect the user to ItemInfo.kt when pressed.

The top bar has a settings button which will redirect the user to AppSettings.kt when pressed. The bottom bar at the bottom of the screen contains three IconButton, Calendar, Lists, and Pantry. When the user clicks any of these they are directed to Calendar.kt, MainMenuLists.kt, and Pantry.kt respectively.

Pantry:

Wire-frame



Light mode



Dark mode



The Pantry screens are largely identical to their wire-frame counterparts, with the only difference being the addition of the settings icon in the top right instead of another kebab menu.

Each item on the screen, obtained from the Room database, is an `ElevatedCard`, displaying the item's name and details (if they have been filled in). Each item can be tapped, bringing the user to the `ItemInfo.kt` screen for that item.

Each item also has a kebab menu, which displays a `DropDown` menu when pressed which contains a `TextButton` labelled "Delete". When Delete is pressed, the item is deleted from the pantry and any lists it's in.

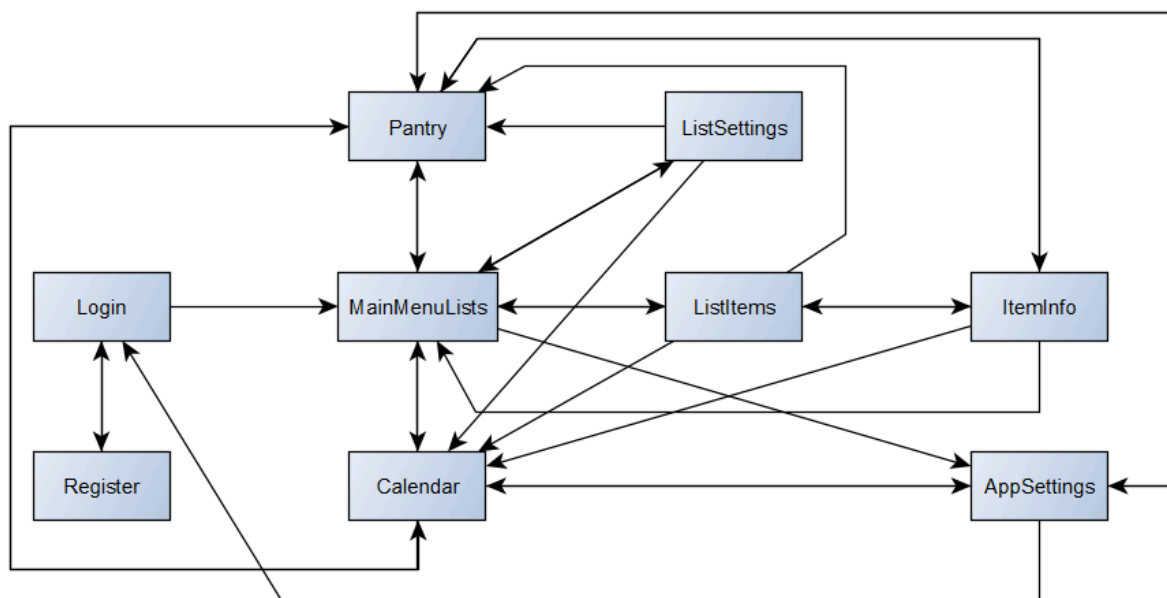
The plus `IconButton` in the top app bar allows the user to create a new item. It begins by showing a `Dialog` with a `TextField` in which the user can enter their new item's name. When the "Add" button is pressed, the inputted name is trimmed and checked to make sure it isn't empty or doesn't already exist, then a new item is created and added to the screen.

The top bar has a settings `IconButton` which will redirect the user to `AppSettings.kt` when pressed.

The bottom bar at the bottom of the screen contains three IconButton, Calendar, Lists, and Pantry. When the user clicks any of these they are directed to Calendar.kt, MainMenuLists.kt, and Pantry.kt respectively.

Navigation

The navigation is all found within the AppNavigation Kotlin file in the screen folder. In our app, the navigation is implemented using NavController and Navhost. The NavController controls the navigation activities while the navhost keeps track of navigation routes, the screens that exist, and their composables to navigate to them. There is a navigation composable for each screen in the app. The starting destination for the navigation is the login page, as this is the first screen the user opens the app to. Each composable is contained within the navHost and can be called using the NavController in other screens to run and navigate to another screen. All of the view models are passed into the app navigation as parameters, and are then passed into each screen composable as needed. Below is a diagram for each screen of the navigation routes the user can take from that screen.



Every navigation composable has a specific screen composable that they call. Each screen composable has arguments of the NavController and the view model which they need. Some of the composables have data passed in to them as they need to display the name of the item or list, and this method is more efficient than making a sql query to get said information. For example, the item info screen composable we wanted to display the item name that they clicked, so we passed the item name between the screens navigations. A screenshot of the code can be seen below.

```
//Item info screen
//Parameters - item id and item name
composable(Routes.itemInfo + "{item_id}" + "{item_name}") {
    //Gets the item id and item name parameters
    val item_id = Integer.parseInt(s = it.arguments?.getString( key = "item_id"))
    val item_name = it.arguments?.getString( key = "item_name")
    ItemVm.setItemId(item_id) //Sets the item id in the ItemViewModel

    //Runs the item info screen composable - arguments the navController, the item name and ItemViewModel
    itemInfoScreen(navController, name = item_name, viewModel = ItemVm)
}
```

To make the naming of each composable easier to call each time we want to navigate to a page, we created a Routes object which stores all the string variables for each composable. This means that when we want to navigate to a page, we can grab the string name of the route/screen from there and eliminate any potential typos or mistakes. An example of this Routes object in use can be seen below:

```
IconButton(onClick = { navController.navigate(Routes.mainmenuLists) },
```

The final part of the navigation is setting the variables the viewmodels need to run. These variables are user ID (used by all view models), the list ID (used by the shopping list view model), and the item ID (used by the item view model). All of the view models contain a private variable that stores these variables and a set function for each variable it requires.

Our app relies heavily on what the user has entered and showing the specific list and items that belong to the user. We don't want other user's lists and items showing up on screen. This is where saving the user's ID across the app is necessary. Continuously passing the user ID through each screen would likely be over-complicated and insecure, so we opted instead to have the user ID set across all view models. To do this, a function was created called setUser(userId: Int), seen below:

```
//Sets the user id for the user that is logged in in the viewModels and the user id variable
fun setUser(userId : Int) {
    usersVm.setUserId(userId)
    ListVm.setUserId(userId)
    pantryVm.setUserId(userId)
    ShoppingListVm.setUserId(userId)
    ItemVm.setUserId(userId)
}
```

This function takes in user ID as a parameter and runs all the view model's setUsedId function passing in this user ID . This allows us to set the user ID in all of the view models and, by doing this, the user ID can be then used inside all functions that require it. It is used in the login composable, when the user logs in, and the app settings composable, when the user logs out or deletes their account.

When the user logs in, the login screen composable called from the login navigation composable has a callback with the user ID as a parameter. This is passed in to the setUser function, setting the user ID across all viewmodels. Once this is done, it navigates the user to the main menu lists screen. This function can be seen below:

```
//Runs the settings screen composable passing in the navController
//onLoggedOut - when the user clicks the log out button
settingsScreen(navController, viewModel = usersVm, user_id = usersVm.getUserId(), onLoggedOut_Delete = {

    //Sets user id to -1 to indicate logged out
    setUser( user_id = -1)

    //Navigates to the login screen
    navController.navigate(Routes.Login)
})
```

The logging out the user/ deleting account has a nearly identical process to the logging in. The appSetting screen composable has onLoggedOut_delete which has no parameters and calls the setUser function with argument -1 and sets user_id to -1 (its default value when not set). It finishes off by then navigating the user to the login page. This function can be seen below:

```
composable(Routes.Login) { //Login Screen

    //Runs the login's screen composable - arguments the navController
    //OnLoggedIn - when the user logs in, sets the user id and navigates
    loginScreen(navController, viewModel = usersVm, onLoggedIn = {
        //Sets the user id to the user's id that is logging in
        setUser( user_id = it)
        //Navigates to the main menu with their shopping lists screen
        navController.navigate(Routes.mainmenuLists)
    })
}
```

The list ID is needed for the shopping list view model to know what list it is looking at. It is set when the composable for list settings and list items are called. In these navigation composables, the list ID is passed in when called and is then retrieved and passed in as an argument to the the setListId function from the ShoppingListViewModel to set the list id. The item ID follows a very similar pattern, with the item ID being passed into the item info composable and is then retrieved and passed in as an argument to the setListId function from the ShoppingListViewModel to set the Item id. Below are screenshots of the list items and list settings navigation composables.

```
//List items screen
//List id and the name of the list is passed through as parameters
composable(Routes.listItems + "{list_id}" + "{name}") {
    //Gets the list id and list name parameters
    val list_id = Integer.parseInt( s = it.arguments?.getString( key = "list_id"))
    ShoppingListVm.setListId(list_id) //Sets the list id in the ShoppingListViewModel
    val list_name = it.arguments?.getString( key = "name")

    //Runs the list Items screen composable - arguments are the navController, the ShoppingListVm
    listItemsScreen(navController, viewModel = ShoppingListVm, list_name)
}
```

```
composable(Routes.listSettings + "{list_id}" + "{name}") { //List settings screen
    //Gets the list id and list name parameters
    val list_id = Integer.parseInt( s = it.arguments?.getString( key = "list_id"))
    ShoppingListVm.setListId(list_id) //Sets the list id in the ShoppingListViewModel
    val list_name = it.arguments?.getString( key = "name")

    //Runs the list Items screen composable - arguments are the navController, the ShoppingListVm
    listSettingsScreen(navController, sViewModel = ShoppingListVm, list_name)
}
```

Room Database

The room database in our app is made up of 4 entities, each with their own DAO. Each DAO has functions with sql queries that can be called and used from the view models. The app follows the architecture of MVVM - Model, View, ViewModel.

Entities:

Lists

This stores all the lists in the database

Columns

- list_id the ID of the list (Primary Key auto generated)
- title the title of the list
- user_id the ID of the user who owns the list
- checked whether or not the checkbox for the list is ticked or not

Foreign key

- From user_id column to the user_id column in the Users entity

This makes sure that no users get added to the Lists entity that don't exist.

Pantry:

This stores all the items in the database

Columns

- item_id the ID of the item (Primary Key auto generated)
- name the name of the item
- expiry the expiry date of the item
- tag the tag on the item
- quantity how many of the item that the user has
- user_id the ID of the user that the item belongs to
- checked whether or not the checkbox of the item is ticked or not
- latitude the latitude coordinate value for the item's shop saved using the GPS
- longitude the longitude coordinate value for the item's shop saved using GPS

Foreign key

- From its user_id column to the user ID column in the Users entity

This makes sure that no users get added to the Pantry entity that don't exist

Constraints

The columns user_id and name must be unique together - allows the users to add the same item name but not the same user twice

ShoppingList:

This is a junction table. Stores the link between the lists and panty entities. It links the list to the items in that list.

Columns

- list_id stores the ID of the list
- item_id stores the ID of the item in the list

Composite Primary key made up of list_id and item_id

Foreign key

- From the list_id column to the list_id column in the Lists entity
- From the user_id column to the user_id column in the Users entity.

This makes sure that no items or lists get added to the entity that don't exist. These are used to join the tables together so that in the sql queries we can get the items that are in a specific list.

Users:

This stores all the users in the database

Columns

- user_id the ID of the user (Primary Key auto generate)
- username the username of the user
- email the email of the user
- password the password of the user

DAO:

ListsDao:

Holds all the sql commands for accessing, updating, and editing the data in the Lists entity

Methods:

getALLlists(user_id:int) : Flow<List<Lists>>

- Used to get all the users lists
- Parameter: The user ID of the user that is signed in
- Returns: A Flow list of Lists entity instances
 - It is a flow so we can retrieve live updates from the database as lists get added for that user

insert(lists : Lists)

- Used to insert a new list
- Suspend function so that it doesn't block the UI and is run in a co-routine in the background

- Parameter: the new list instance being inserted

`delete(lists:Lists)`

- Used to delete a list
- Suspend function so that it doesn't block the Ui and is run in a co-routine in the background
- Parameter: the list instance to be deleted

`updateCheckbox(check : Boolean, list_id: Int, user_id: Int)`

- Used to update the checked value for a specific user list
- Suspend function
- Parameters: the value the checkbox is being set to whether it is checked or not, the list ID and the user id

`amountItems(list_id : Int) : Int`

- Used to count the amount of items in a user's list so we can display it
- Suspend function
- Parameters: the list ID of the list we are gonna count the items of
- Return: the amount of items in the list

PantryDao:

Holds all the sql commands for accessing, updating and editing data in the Pantry entity

Methods:

`getAllItems(user_id : Int) Flow<List<Pantry>>`

- Gets all the user's items in the pantry that are checked - for an item to be shown in the pantry it's checkbox must be checked
- Used on the pantry screen to show all the user's items they have
- Parameters: the signed in user id
- Return: a flow list of pantry entity instances - it is a flow so we can retrieve live updates from the database as new checked items appear in the pantry

`getDates(user_id : Int): List<String>`

- Suspend function
- Gets all the expiry dates of user's items that have a expiry set
- Used in the calendar screen to know when items are expiring and display a red line on that date
- Parameters: the user's id
- Return: a list of strings - all the expiry dates

`getExpiryItems(user_id : Int, expiry : String): List<Pantry>`

- Suspend function

- Gets all the user's items that are expiring on a certain date
- Used in the calendar screen to display the items expiring on that day
- Parameters: the user ID and the expiry date we are looking at
- Return: a list of pantry entity instances - the items that expire on that date

insert(items:Pantry) : Long

- Inserts an item into the pantry
- Suspend function
- Has a on conflict strategy of ignore if there is a conflict - this means it will not throw an exception if there is an error but instead return -1
- The ignore returning -1 is used for checking if an item already exists
- Parameter: the new item the user is inserting
- Return: the row ID (inserted) or -1 (error/conflict)

delete(items: Pantry)

- Deletes an item from the pantry
- Suspend function
- Parameter: the item that the user wants to delete

getItemId(name : String, user_id: Int) : Long

- Gets the item ID of a user's item
- Suspend function
- Used when adding an item in a list to insert it into the shopping list entity
- Limited to getting 1 row
- Parameter: the name of the item and the user id
- Return: the item ID for the item

getItemInfo(item_id : Int, user_id : Int): Pantry

- Gets all the info for a user's item
- Used in the item info screen to display its values
- Suspend function
- Parameters: the item ID and the user id
- Return: a Pantry instance - all the info about the item

updateItem(quantity: Int, expiry : String, tag : String, item_id: Int, user_id: Int)

- Updates the quantity, expiry and tag of an item
- Used in the item info screen to update the values
- Suspend function
- Parameters: the new quantity, tag, and expiry data, item ID, and user id

updateQuantity(quantity: Int, item_id: Long, user_id: Int)

- Updates the quantity of an item
- Suspend function

- Parameters: the new quantity, item ID, and user ID

updateCheckbox(checked: Boolean, item_id: Long, user_id: Int)

- Updates the checked value for a user's item - the state of the checkbox
- Suspend function
- Used to keep track if the checkbox is checked or not
- Parameters: the value the checkbox is being set to whether it is checked or not, item ID, and user id

updateLocation(latitude: Double, longitude : Double, item_id: Int, user_id: Int)

- Updates the latitude and longitude values for the user item's shop
- Used to keep track of the location of the shop
- Suspend function
- Parameters: latitude value, longitude value, item ID, and user id

ShoppingListDao:

Holds all the sql commands for accessing, updating and editing data in the Pantry entity

Methods

getAllListItems(list_id: Int): Flow<List<Pantry>>

- Gets all the items in a shopping list
- Parameters: the ID of the list
- Return: a flow list of pantry entity instances - a flow list so that we can retrieve live updates from the database as new items appear in the list

getPantryItems(list_id: Int, user_id: Int) : Flow<List<String>>

- Gets all the user's items that are not in this shopping list but are in the pantry
- Used for the dropdown menu when adding an existing item in the shopping list screen
- Parameters: the list ID and the user id
- Return: a flow list of strings - it is a flow list so that we can retrieve live updates from the database as new items appear in other lists or the pantry

getPantryItemId(name: String, user_id: Int) : Int

- Gets the ID of an item
- Suspend Function
- Used to add a existing item to the a list - adding to the junction table
- Parameters - name of the item and the user ID
- Return: an int representing the item ID

insert(shoppingList : ShoppingList)

- Inserts a link between a list and an item - adds a item to the list

- Suspend Function
- Parameter: the new Shopping list instance being inserted

delete(shoppingList: ShoppingList)

- Deletes the link between a list and an item - removing an item from a list
- Suspend Function
- Parameters: the shoppingList instance being removed

UsersDao:

Holds all the sql commands for accessing, updating and editing data in the Users entity

Methods

insert(users: Users) : Long

- Inserts a user into the users entity
- Used for register when creating a new user
- Suspend function
- Parameter: the new user instance being added
- Return: a Long - the row ID of the user to signify it was added

delete(user_id: Int)

- Deletes a user from the users entity based on the user's id
- Used for deleting a user's account
- Suspend function
- Parameter: the user ID of the user being deleted

findUser(username : String, password : String): Users?

- Gets the user's info based on the username and password
- Used for login to see if user exists
- Suspend function
- Parameters: the username and password of the user
- Return: a user instance (if the user exists) or null (if the user doesn't exist)

checkUsername(username : String): String?

- Gets the username of a user based on a username
- It is used for register to see if the username already exists
- Suspend function
- Parameter: the username to look for
- Return: string (if the username exists) or null (if the username doesn't exist)

getUser(user_id : Int) : Users

- Gets all the information about a user
- Used for app settings to display the username and email

- Suspend function
- Parameter: the user id
- Return: a Users instance

updateUsernameEmail(newUsername: String, newEmail : String, user_id : Int)

- Updates the username and email of a user
- Used to edit the username and email of a user in the app settings
- Suspend function
- Parameters: new username, new email and user's id

userExists(username: String, email: String) : Users/

- Checks if a user exists in the database based on username and email
- Used in forgotten password to check if their account exists
- Suspend function
- Parameters: the username and email
- Return: a user instance (if the user does exist) and null (if the user doesn't exist)

updatePassword(newpassword:String, user_id: Int)

- Updates the password of a user
- Used in forgotten password to change a user's password
- Suspend function
- Parameters: new password and user's id

View Models

In our app, we have 5 view models: ListViewModel, PantryViewModel, ShoppingListViewModel, ItemViewModel, and UsersViewModel. They are all created in the main activity, meaning that they will be linked to the activity lifecycle, will live as long as the activity lives, and will survive recomposition. These view models are then passed into the navigation composable as arguments and then passed into the screens that use them. Each screen requires a certain view model to function. Each view model has variables that they save, and functions to set these variables from the navigation. All of them save the ID of the signed in user, which is set to -1 when no user is signed in. These saved variables are key to the functioning of the view models, and are passed into the DAO's when they are called from the viewmodel.

ItemViewModel:

The ItemViewModel is passed into and used in the ItemInfo screen. It handles all the functions for access, editing and updating data for that screen. It has one parameter which is the Pantry Dao. All of its sql queries are to the Pantry entity, therefore all functions that it calls come from the PantryDao. It saves the ID of the item the user wants to look at and the user id.

Methods

setItemId(newItemId : Int)

- Sets the item id
- Parameter: the ID of the item the user want to look at

setUserId(newUserId: Int)

- Sets the user id
- Parameter: the ID of the user that is signed in

getItemInfo() : Pantry

- Gets all the info about an item
- Used to get all the info of the item to display it
- It calls the getItemInfo suspend function and passes in the item ID and user ID that is saved
 - This returns a pantry instance and saves it in a value.
- Suspend function - so it could return a pantry instance
- Return: the value that was got from the getItemInfo function - a pantry entity instance

updateItem(new_quantity : Int, newDate : String, newTag : String)

- This updates the quantity, name and tag of an item
- Used in item info to save changes the user makes

- It calls the updateItem suspend function in a coroutine using viewModelScope.launch so that it runs in the background and it doesn't freeze the UI
- Passes in the new quantity, new date and new tag into the function
- Parameters: new quantity, new date and new tag of the item

updateLocation(latitude: Double, longitude: Double)

- Updates/Saves the longitude and latitude values of an item's shop
- Used to save the location of a shop for an item gotten by the gps
- Parameters - the latitude and longitude values gotten from the gps
- Calls the updateLocation suspend function in a coroutine using viewModelScope.launch
- Passes in the latitude, longitude, the item ID and the user ID that are saved

deleteItem(item : Pantry)

- Deletes an item from the pantry
- Used when the user wants to delete an item, in the delete item button and the accelerometer for shake to delete
- Calls the delete suspend function in a coroutine using viewModelScope.launch
- Passes in the pantry instance parameter that is to be deleted
- Parameter: the pantry instance that the user wants to delete

ListsViewModel

The ListsViewModel is passed into and used in the MainMenuLists screen and handles all the functions for accessing, editing, and updating data needed for that screen. The ListsDao is its only parameter. All of its sql queries are to the Lists entity, therefore all the functions that it calls come from the ListsDao.

It has two variables: the user ID and a flow list of Lists entity instances called lists. These are used to store all the lists the user has so we can collect it on the screen and display them. It runs the getAllLists function and passes in the user id.

Methods

setUserId(newUser_id: Int)

- Sets the user ID and lists to the getALlists function but with the new user ID passed
- Parameter: the ID of the user that is signed in

addList(title : String)

- Adds a list to the Lists entity
- Used to add a new list
- It trims down the name of the new list to remove leading and trailing spaces

- Checks to make sure it is not empty and it it is returns
- Calls the insert suspend function in a coroutine using `viewModelScope.launch`
- Passes in the name of the new list, the user ID and the value of false to set the checkbox to be unchecked
- Parameter: the name of the new list

`deleteList(list: Lists)`

- Delete a list from the Lists entity
- Used to delete a list
- Calls the delete suspend function in a coroutine using `viewModelScope.launch`
- Passes in the Lists entity instance that is to be deleted
- Parameter: the Lists entity instance that is being deleted

`listcheckboxbox(checked: Boolean, list_id: Int)`

- Updates the checkbox value for a list to be checked or not
- Used when the user checks or unchecks a checkbox of a list
- Calls the updateCheckbox suspend function in a coroutine using `viewModelScope.launch`
- Passes in whether the box is to be updated to checked or unchecked, the list ID and the user_id
- Parameters: boolean to say whether the box is to be updated to checked or unchecked, and the list id

`amountItems(list_id : Int) : Int`

- Gets the amount of items in a list
- Used to display the quantity in a list
- Calls the amountItems suspend function in a coroutine using `viewModelScope.launch` and stores the value that is returned in a value called amount
- Parameters: the list id
- Return: amount of items in the list

PantryViewModel

The `PantryViewModel` is passed into and used in the calendar screen and the pantry screen. It handles all the functions for accessing, editing, and updating data needed for these screens. Its one parameter is the `PantryDao`. All of its sql queries are to the `Pantry` entity, so all the functions that it calls come from the `PantryDao`.

It has two variables: the user ID and a flow list of `Pantry` entity instances called `items`. These are used to store all the items the user has got in the pantry so we can collect it on the screen and display them. For an item to be seen as in the pantry, it

must have its checked value as true (meaning its checkbox has been ticked). It runs the getAllLists function and passes in the user id.

Methods

setUserId(newuser_id: Int)

- Sets the user ID and items to the getAllLists function but with the new user ID passed
- Parameter: the ID of the user that is signed in

getDates(): List<String>

- Gets all the expiry dates for the user's pantry items that have an expiry set
- Suspend function
- Used in the calendar screen to know if a date on the calendar has an item expiring and to display a red line on it to indicate it
- Calls the getDates suspend function and passes in the user id
- Return: the list of the dates

getExpiryItems(expiry: String) : List<Pantry>

- Gets all the pantry items expiring on a certain date
- Used in the calendar when the user clicks a date with items expiring that day so it can display them
- Suspend function
- Calls the getExpiryItems suspend function and passes in the user ID and the expiry date parameter
- Parameter: the expiry date we want to get items expiring on
- Return: the list of Pantry entity instances with an expiry of the date passed in

deleteItem(item:Pantry)

- Delete an item from the Pantry entity
- Used to delete an item from the lists and the pantry
- Calls the delete suspend function in a coroutine using viewModelScope.launch
- Passes in the item to be deleted
- Parameter: the Pantry entity instance, the item, to be deleted

checkbox(checkered: Boolean, item_id: Long)

- Updates the checked value to be checked or not in the database
- Used when a user checks or unchecks a checkbox of an item
- Calls the updateCheckbox suspend function in a coroutine using viewModelScope.launch
- Passes in what the checkbox is being updated to, the item ID and the user id
- Parameters: boolean to say whether the box is to be updated to checked or unchecked, and the ID of the item we are updating

addItem(name: String) : Boolean

- Adds an item to the database
- Suspend function - so it can return a value
- Trims the name to remove leading and trailing spaces
- Calls the insert suspend function and passes in a new Pantry entity instance with the name and user ID set and the rest of the values set to be default values
- Parameter: the name of the new item
- The value that is returned by this function is saved in a variable called check
- It then checks to see if -1 or a row ID was returned and if a row ID was returned returns true and if -1 it returns false - this checks to see if an item already exists and lets the screen know if it does by returning true or false

ShoppingListViewModel

The ShoppingListViewModel is passed into and used in the list items and list settings screen. It handles all the functions for accessing, editing, and updating data needed for these screens. It has two parameters: the ShoppingListDao and the PantryDao. Its SQL queries are mostly to the ShoppingList entity, with some also needing the Pantry entity. All of the functions will be called from the ShoppingListDao unless stated it's from the PantryDao.

It has four variables: the user id, the ID of the list the user wants to look at, a flow list of Pantry entity instances called listItems, and a flow list of strings called pantryItems. listItems is used to store all the items the user has in this list, and pantryItems is used to store all the items the user has but in other lists. These are used in a dropdown when adding a new item. listItems runs the getAlllistItems function and passes in the list id. pantryItems calls the getPantryItems function and passes in the list ID and user id.

Methods

setUserId(newUser_id: Int)

- Sets the user ID
- Parameter: the ID of the user that is signed in

setListId(newlist_id: Int)

- Sets the list id
- Sets listItems to the getAlllistItems function but with the new list id
- Sets pantryItems to the getPantryItems function but with the new list ID and user id
- Parameter: the ID of the list the user wants to look at

addListItem(name: String) : Boolean

- Adds an item to the database and to the list

- Used to add an item to a list
- Suspend function - so it can return a value
- Trims the name to remove leading and trailing spaces
- Calls the insert suspend function from the PantryDao and passes in a new Pantry entity instance with the name and user ID set and the rest of the values set to be default values
- The value that is returned by this function is saved in a variable called check
- It then checks to see if -1 or a row ID was returned.
- If a row ID was returned it calls the getItemId() suspend function from the PantryDao, passes in the name of the new item and the user ID and stores the returned value of that in a value called itemId.
 - This gets the item ID of the new item so we can then insert it into the ShoppingList entity.
 - It then calls the insert function and passes in a new ShoppingList entity instance with the list ID and the item ID as an integer.
 - It finishes by returning true to signify it was added and the item didn't exist already
 - If -1 is returned it means an item with that name already exists so it returns false back to let the screen know
- Parameters: the name of the new item

addExistingItem(name : String)

- Adds an item that does not exist in this list but does exist in another
- Used in when the user wants to add an item to a list they can select an existing item from a dropdown menu
- Calls the getPantryItemId suspend function in a co-routine using viewModelScope.launch and saves the value returned in a value called itemId
- Passes in the name of the existing item and the user id
- It then calls the insert suspend function and passes in a new ShoppingList entity instance with the list ID and the item ID of the existing item as an integer
- Parameter: the name of the existing item

deleteListItem(item_id : Int)

- Deletes the link between a list and an item, a ShoppingList entity instance
- Used to delete an item from a list
- Calls the delete suspend function in a coroutine using viewModelScope.launch
- Passes a ShoppingList entity instance with the list ID and the item ID of the item being deleted
- Parameter: the item ID of the item being deleted

checkbox(checkered: Boolean, item_id: Long)

- Updates the checked value to be checked or not in the database

- Used when a user checks or unchecks a checkbox of an item
- Calls the updateCheckbox suspend function from the PantryDao in a coroutine using viewModelScope.launch
- Passes in what the checkbox is being updated to, the item ID and the user id
- Parameters: boolean to say whether the box is to be updated to checked or unchecked, and the ID of the item we are updating

updateQuantity(newQuantity : Int, item_id: Long)

- Updates the quantity of an item
- Used to update the quantity of an item to 1 when the item gets checked and has a quantity of 0
- Calls the updateQuantity suspend function from the pantryDao in a coroutine using viewModelScope.launch
- Passes in the new quantity, the item ID and the user id
- Parameter: the new quantity and the ID of the item we are updating

UsersViewModel

The UsersViewModel is passed into and used in the login, register, and app settings screens. It handles all the functions for accessing, editing, and updating data needed for these screens. It has one parameter which is the UsersDao. All of its SQL queries are to the Users entity so all the functions that it calls come from the UsersDao. It only has the user ID variable.

Methods

setUserId(newUserId: Int)

- Sets the user ID
- Parameter: the ID of the user that is signed in

getUserId(): Int

- Used in the app navigation when calling the settingsScreen composable for app settings as it requires the user ID to be passed in
- Return: the user ID variable

getUser(user_id : Int) : Users

- Gets the Users entity instance based on the user id
- Used to get the user's information so it can display the username and email
- Suspend function
- Calls the getUser suspend function, passes in the user ID parameter and stores the returned Users entity instance by it in a value called user
- Parameter: the user's id
- Return: the User entity

updateUsernameEmail(newUsername: String, newEmail: String, user_id: Int)

- Updates the username and email of the user
- Used if the user changes the username or email and clicks the save button
- Calls the updateUsernameEmail suspend function in a coroutine using ViewModelScope.launch
- Passes in the new username, the new email and the user id
- Parameters: new username, new email and ID of the user we are updating

updatePassword(newPassword: String, user_id: Int)

- Updates the password of a user
- Used if the user forgets their password and needs to change it
- Calls the updatePassword suspend function in a coroutine using viewModelScope.launch
- Passes in the new password and the user id
- Parameters: the new password and the ID of the user that we are changing

userExists(username: String, email: String): Users?

- Check if a user exists based on email and username
- Used to check the user is changing the password for their account
- Suspend function
- Calls the userExists suspend function, passes in the username and email and saves the returned Users instance or null in the value user
- It then checks to see if user is null, the username is empty, or the email is empty and if any of these are true returns null
 - If it they aren't, it will return the user instance
- Parameters: the username and email of the user
- Returns: User (if the user exists, and the username and password are correct), null (if the user doesn't exist, or the username or password aren't correct)

addUser(user: Users): Long

- Inserts a user into the database
- Used when a new user registers
- Suspend function
- Calls the insert suspend function, passes in the user instance and returns the row ID of the user
- Parameter: the new Users instance being inserted
- Return: The row ID of the new user

deleteUser(user_id: Int)

- Deletes a user from the database
- Used when the user wants to delete their account
- Calls the delete suspend function in a co-routine using viewModelScope.launch
- Passes in the ID of the user to be deleted

- Parameter - the ID of the user to be deleted

checkUsername(username: String): Boolean

- Checks to see if the username already exists in the database
- Used when the user is registering to make sure they don't register with the same username as someone else
- Suspend function
- Calls the checkUsername suspend function and passes in the username
 - If it returns null (signifying the username doesn't already exist) false will be returned, and if it does not return null true will be returned
- Parameter: username
- Return: true if username is correct, false if it isn't

checkUser(username: String, password: String) : Users?

- Checks to see if the user already exists in the database
- Used when the user logs in to make sure it is correct
- Suspend function
- Calls the findUser suspend function, passes in the username and password, and saves the value that is returned in a value called user
- If user is equal to null, username is empty, or password is empty, null will be returned to signify login is incorrect, otherwise user is returned
- Parameters: username and password of the user
- Return: User if login details are correct, null if they aren't

Sensors

GPS

- Used in ItemInfo.kt to save the location of a shop for an item. Saves the current location of the user as the shop and stores the latitude and longitude values in the database for that item
- When the button to save the location is clicked, it will add the location listener. It asks the user for permission to use the location services, and they can either deny or allow it.
 - If they deny permissions, a pop up will display to let them know why they are needed and that their choice can be changed in settings
 - If they allow permission, the location listener will be added, tracking will begin and the started tracking variable that keeps track of whether it's tracking or not will be set to true
- Once the longitude and latitude values have been retrieved the location listener is removed, tracking is stopped, the started tracking variable is set to false, and the updateLocation function from the ItemViewModel is run with the latitude and longitude values to save the location.
- While waiting for the GPS to get the longitude and latitude a circular progress indicator appears to let the user know that it is working.

Accelerometer:

- Called in ItemInfo.kt by the shakeToDelete() function, allowing the user to delete an item in their list by shaking their phone instead of pressing the button

```
shakeToDelete(navController, viewModel, itemInfo, name)
```

- Creates sensorManager, sensorShake (which is used to get access to the phone's accelerometer), and listener to sense when the phone is shaken
- When the accelerometer registers a change in acceleration along its axes, it measures the acceleration of each axis
- If the acceleration is greater than 40m/s^2 across all axes combined, the app registers this as shaking and a boolean is switched to show a Dialog
- The sensor delay is set to NORMAL as it doesn't need to be checked often, and the sensor listener is turned off when the Disposable effect is complete

Intents

Map Intent

- Used in itemInfo.kt once the shop for the item has been saved. When the shop for an item has been saved, a button that will show the location was saved will appear alongside a reset button to save a new shop.
- When the saved shop button is clicked, the longitude and latitude values are passed into a map intent with text “Shop for (item name)” and sends that to an external map.
- Once the intent is created startActivity starts the map intent and will open an external map drop a pin on the location of the shop that was saved and display the text

```
Button(  
    onClick = {  
        //Map intent that uses puts the longitude and latitude saved into a map and shows it  
        val mapIntent: Intent =  
            "geo:0,0?q=${latitude},${longitude}(Shop+for+${itemInfo.name})".toUri()  
            .let { location ->  
                Intent( action = Intent.ACTION_VIEW, uri = location)  
            }  
        this_context.startActivity(mapIntent) //starts the intent  
    },  
    modifier = Modifier.defaultMinSize( minWidth = 48.dp, minHeight = 48.dp)  
 ) { Text( text = stringResource( id = R.string.see_shop), fontSize = 14.sp) }
```

Email Intent

- Used in ListSettings.kt to share a list and its items through email to another person. When the user clicks the button to share their list a dialog box will appear prompting them to enter the email of the recipient. This email is then trimmed and validated, and the email intent starts to be created.
- The email intent starts with first setting the recipient email that was entered, then setting the subject of the email to the list name, and finally the body is inserted.
- The email body is created using the function compose_email(listItems). This function loops through all the items in the list creating a string that contains all the item's name and quantity and their tag if they have one. This is then returned back and saved in the email_body variable and inserted into the email body in the email intent.
- Once the intent is created, the startActivity runs the intent and the user chooses what app to open
 - Once they open an email app it fills in the recipient email, subject, and body of the email, and all they have to do is click send.

```
val t = email.trim()  
if (t.isNotEmpty()) {  
    email = t  
    show_dialog = false  
    email_body += compose_email(listItems)  
  
    val sending = Intent( action = Intent.ACTION_SEND).apply {  
        type = "text/plain"  
        putExtra( name = Intent.EXTRA_EMAIL, value = arrayOf(email)) // recipient  
        putExtra( name = Intent.EXTRA_SUBJECT, value = "${list_name}")  
        putExtra( name = Intent.EXTRA_TEXT, value = email_body)  
    }  
  
    context.startActivity( intent = sending)
```

Validation file

The validation kotlin file is used for registration, password resetting, changing email and/or password in app settings, entering recipient email to share your list with, and entering expiration dates. This file provides the validation for email, username, password, re-entered password, and dates. Each one has their own validation function which checks to see if each one meets specific criteria. All of the functions return a string which are the errors that will be displayed if not valid. An empty string returned means no error occurred and they are valid.

`emailValidate(email: String) : String`

This is the email validation function. It has a parameter of the email it is checking. The criteria the email must meet to be valid is that the email must not be empty and it must match the pattern of an email address.

`passwordValidate(password: String) : String`

This is the password validation function. It has a parameter of the password it is checking. The criteria the password must meet to be valid is that it must not be empty and must be at least 6 characters.

`repasswordValidate(password: String, repassword: String) : String`

This is the re-entered password validation function. It has parameters of the password that was entered and the re-entry password. The criteria for the repassword is that it must not be empty and it must match the password.

`usernameValidate(username: String, check: Boolean) : String`

This is the username validate function. It has parameters of the username that is being checked and a Boolean that is returned from the checkUsername function from the UsersViewModel checking if the username already exists. The criteria the username must meet is that it must not be empty, must be at least 4 characters, and the check value must be false (meaning the username doesn't already exist).

`validateInputs(username: String, check: Boolean, email: String, password: String, repassword: String) : Boolean`

This is used to validate all inputs are correct when the user clicks the register button to make sure that everything is valid and that all have been validated. It has parameters for all the values for all the validation functions. In this function it has an if statement that makes sure all validation functions return an empty string and if they do return back true allowing the register page to continue and save the user otherwise false making the register page display an error.

`dateValidate(date: String) : String`

This is used for validating the date that is entered in the date field in the item info is a valid date. It checks to make sure that it matches a certain date pattern using Regex

and if it does, it returns an empty string and if not returns an error string which gets displayed on the item info screen.

Conclusion

Overall we feel the app we developed achieved the shopping list app we set out to create and improves and speeds-up the process of creating and maintaining shopping lists. Though some features were either removed due to difficulties with getting it set up or time constraints, we feel the features that we implemented are key to the shopping list app and help it achieve its main goal. If we were to continue developing the app, we feel there are new features and improvements we could make. We feel the features that didn't make it in like the map, notifications for soon-to-expire items, repeating lists, and a button to remove lists in list settings could definitely be added while also making improvements such as allowing the user to input how much of an item they want to get vs what they currently have, removing items past their expiry dates, and allowing the user to choose a unit by which to measure the quantity of an item.

Overall while developing the app, we learned a lot about modern android app development. We got hands-on experience with jetpack compose, viewmodels, room database, and navigation components and got to see how everything works together and manage recomposition correctly. It has provided us with invaluable experience on developing an android app and furthered our understanding of android and mobile software development as a whole.